CP386: Assignment 2 – Fall 2026

Due on Oct 26, 2026 (Before 11:59 PM)

This is a group (of two) assignment, and we will practice the concept of the parent-child process, inter-process communication, and some related system calls.

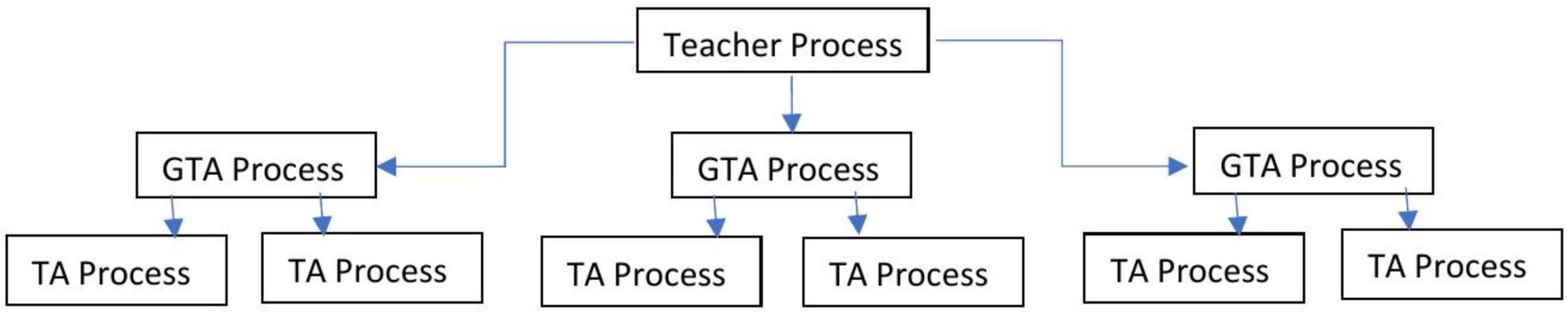
General Instructions:

- For this assignment, you must use C language syntax. Your code must compile using **makefile without errors.** You will be provided with a Makefile and instructions on using it.
- **Test your program thoroughly with the GCC compiler in a Linux environment.**
- If your code does not compile, **then you will score zero**. Therefore, ensure you have removed all syntax errors from your code.
- The Gradescope platform will be used to upload the assignment file(s) for grading. The link to the Gradescope assignment is available on Myls course page. Drag and drop your code file(s) for submission into Gradescope. **Ensure your file name is as suggested in the assignment; using a different name may score Zero.**
- Please note that the submitted code will be checked for plagiarism. By submitting the code file(s), you would confirm that you have not received unauthorized assistance in preparing the assignment. You also confirm that you are aware of course policies for submitted work.
- Marks will be deducted for any questions where these requirements are not met.
- Multiple attempts will be allowed, but only your last submission before the deadline will be graded. We reserve the right to take off points for not following directions.

Question 1

Create a C program (name it "assignment_average.c") that finds the average grade of the course assignments. The teacher gives two assignments for every chapter. You must calculate the average grade for each assignment in all the chapters.

- The program should have a teacher process that reads the grades ("sample_in_grades.txt" grades file is available under the Assignment 1 section on Myls) of all chapters' assignments and creates a two-dimensional matrix of grades.
- Teacher process then creates a "**GradTA**" process. Each "**GradTA**" process takes care of solving a chapter.
- Each "**GradTA**" process would create "**TA**" processes and pass one assignment to each of them. "TA" process would calculate and print the average for that assignment. The process tree is shown below:



The input file "sample_in_grades.txt" contains data for three chapters and two assignments for each chapter (six columns). The first two columns are grades for two assignments for chapter 1, the following two columns are the grades for two assignments for chapter 2, and the last two columns are the grades for two assignments for chapter 3. The Grades file contains grades for 10 students. Make sure your program should dyanamically create GTAs and TAs processes based on the the number of the chpaters (you can assume that each chapter will always have two assingments).

Use makefile to compile the program written above. The instructions to use and contents of the makefile have been provided on the Myls course page. The other implementation details are at your discretion, and you are free to explore.

To invoke the program, first use the command: `./grades sample_in_grades.txt` in the terminal OR use the command: `make runq1` via `makefile`.

The expected output for Question 1:

```
Assignment 1 - Average = 10.200000
Assignment 2 - Average = 11.600000
Assignment 3 - Average = 13.700000
Assignment 4 - Average = 13.600000
Assignment 5 - Average = 9.400000
Assignment 6 - Average = 11.600000
```

Note: When submitting the source code files for Question 1, name them like:

- `assignment_average.c`

Question 2

In C, write a program (name it "process_dispatcher.c") that will involve using the UNIX **fork(), exec(), wait(), dup2(), pipe(), shm_open(), ftruncate(), mmap(), munmap(), and shm_unlink()** system calls. The program creates a multi-level process tree and uses two different IPC mechanisms for distinct purposes: POSIX shared memory to transfer the input workload from a dedicated reader process to the parent, and UNIX pipes to stream command output and return job results through the process hierarchy. The program includes code to accomplish the following tasks (use the described process hierarchy; don't combine these responsibilities into one process):

- A parent process will receive the input filename but must not open or read the file directly. The parent will use `fork()` system calls for creating child processes whenever required and collecting the output from these processes. The following steps must be completed:
    - Creation of a **reader child process** to read the contents of the input file. The input file contains **one job per line**. Each job can contain either a single Linux command or two Linux commands separated by the pipe (|) character. The Reader Child will place the contents of the file into a POSIX shared-memory area.
    - Creation of a **manager child process** for each job. The Manager Child will create the required command-executing child process(es). The command processes will execute Linux commands using `execvp()`. UNIX pipes and `dup2()` must be used to transfer the command output between processes.
    - For a job containing two commands separated by pipe (|), the Manager Child must create two command processes and connect them using a UNIX pipe. The standard output of the first command becomes the standard input of the second command.
    - The Manager Child will collect the final output of the job and return it to the Parent Process through a separate result pipe.
    - The Parent Process will write the result of each job into a file named **"output.txt"**. Only the Parent Process is allowed to write to this file.

The program flow is described below:

- **Reading the Input File**: The command-line arguments must be used by the Parent Process to obtain the input filename. The Parent Process will create a Reader Child Process using `fork()`, and the **Reader Child Process, not the Parent Process, will read the contents of the input file** (for example, `"sample_in_dispatcher.txt"`). The input file contains one job per line. A job can contain a single Linux command, or a command with arguments, or two commands separated by the pipe (`|`) character, e.g., `ps -e | wc -l`. For this assignment, a job can contain a maximum of two commands. You do not need to support more than one | character on a line. You may assume that

command arguments are separated by whitespace. The shell features such as quoted strings, wildcard expansion, environment-variable expansion, command substitution, &&, ||, <, and > do not need to be supported.

- The contents of the input file will be written to a **POSIX shared-memory area** by the Reader Child Process. The shared-memory implementation must use the appropriate POSIX functions such as `shm_open()`, `ftruncate()`, `mmap()`, `munmap()`, and `shm_unlink()`.
- After the complete file contents have been written to shared memory, the Reader Child Process will terminate. The parent process must use `wait()` or `waitpid()` to wait for the Reader Child Process to terminate before processing the contents of the shared-memory area.
- **Copying and Parsing the Shared-Memory Contents:** After the Reader Child Process terminates, the parent process will access the contents stored in the shared-memory area. The contents from shared memory must be copied into a **dynamically allocated array** in the parent process. After the contents have been copied successfully, the parent process must properly unmap and remove the POSIX shared-memory object using the appropriate system calls.
- The parent process will then separate the input into individual jobs. For each job, the parent process must determine whether the job contains a single command or two commands separated by pipe (|). Each command and its parameters must be converted into a NULL-terminated argument array that can be supplied to `execvp()`.
- **Creating a Manager Child Process for Each Job:** The jobs must be executed **one by one in the same order in which they appear in the input file**. For every job, the parent process must, create a **result pipe** using pipe() → create one **Manager Child Process** using fork() → allow the Manager Child Process to create the command-executing child process(es) → read the final result returned by the Manager Child through the result pipe → use wait() or waitpid() to collect the Manager Child Process. Remember parent process itself must **not execute the Linux command using execvp()**.
- **Executing a Job Containing 1 or 2 Commands**: If a job contains only one command, the Manager Child Process will perform the following operations:
    - Create a pipe for receiving the command output and also create another child process, called the **Command Child**, using `fork()`. The Command Child will use `dup2()` to redirect its standard output (STDOUT_FILENO) to the write end of the pipe. The Command Child will execute the Linux command using `execvp(args[0], args);`
    - The Manager Child will read the command output from the read end of the pipe and will use wait() or waitpid() to collect the Command Child and determine its exit status. The Manager Child will send the command output and exit-status information to the parent process through theresult pipe. The parent process will then write the received result into `"output.txt"`.
- If a job contains two commands separated by pipe (|), the Manager Child Process must create a real UNIX pipeline between two separate child processes. The Manager Child will create a **Command Pipe** connecting the first command to the second command; and an **Output Pipe** through which the final output of the second command is returned to the Manager Child. The Manager Child will then create two child processes using fork(). The first process is called **Command Child 1**, that must redirect STDOUT_FILENO to the write end of the Command Pipe using dup2() and execute the first command using execvp(). The second process is called **Command Child 2** will redirect STDIN_FILENO to the read end of the Command Pipe using dup2(), redirect STDOUT_FILENO to the write end of the Output Pipe using dup2(), then execute the second command and write the Output Pipe and read by the Manager Child. The Manager Child must then send the final output and the exit-status information to the Parent Process through the result pipe (note: the use of system(), popen() to create the pipeline is **not allowed**)
- For every job, the Manager Child will return the result of the job to the Parent Process through the result pipe. The Parent Process is responsible for creating and writing to the file `"output.txt"`. The following function will be invoked for each job:

```
static int writeOutput(const char *command, const char *output)
{
    FILE *fp = fopen("output.txt", "a");
    if (fp == NULL) {
        perror("fopen");
        return -1;}
    fprintf(fp, "The output of: %s : is\n", command);
```

```c
        fprintf(fp, ">>>>>>>>>>>>>>>\n%s<<<<<<<<<<<<<<<\n", output);
        fclose(fp);
        return 0;
    }
```

To invoke the program, first use the command: `./process_dispatcher sample_in_dispatcher.txt` in the terminal OR use the command: `make runq2` via `makefile`. The expected contents of output.txt are:

```
The output of: echo Hello_from_single_command : is
>>>>>>>>>>>>>>>
Hello_from_single_command
[Process status]
Stage 1 (echo): exit 0
<<<<<<<<<<<<<<<
The output of: echo one two three four | wc -w : is
>>>>>>>>>>>>>>>
4
[Process status]
Stage 1 (echo): exit 0
Stage 2 (wc): exit 0
<<<<<<<<<<<<<<<
The output of: seq 3 5 | sort -r : is
>>>>>>>>>>>>>>>
5
4
3
[Process status]
Stage 1 (seq): exit 0
Stage 2 (sort): exit 0
<<<<<<<<<<<<<<<
The output of: invalid_command_xyz : is
>>>>>>>>>>>>>>>
[Process status]
Stage 1 (invalid_command_xyz): exit 127
<<<<<<<<<<<<<<<
The output of: echo alpha beta gamma | tr a-z A-Z : is
>>>>>>>>>>>>>>>
ALPHA BETA GAMMA
[Process status]
Stage 1 (echo): exit 0
Stage 2 (tr): exit 0
<<<<<<<<<<<<<<<
The output of: cat /etc/passwd | grep bash : is
>>>>>>>>>>>>>>>
root:x:0:0:root:/root:/bin/bash
[Process status]
Stage 1 (cat): exit 0
Stage 2 (grep): exit 0
<<<<<<<<<<<<<<<
```

Note: When submitting source code files for Question 2, name them like:

- `process_dispatcher.c`